

PROJECT TITLE

In-Depth Analysis of Air Quality Index (AQI) in Delhi Using Python.

1. INTRODUCTION

Air pollution is one of the biggest environmental problems in Delhi. The Air Quality Index (AQI) is used to measure the quality of air and indicate how polluted it is. Poor air quality can lead to serious health issues such as respiratory diseases, asthma, and heart problems. This project analyzes Delhi's AQI data using Python to identify pollution trends, major pollutants, seasonal variations, and their impact on air quality. Data analysis and visualization techniques are used to generate meaningful insights.

2. PROBLEM STATEMENT

Delhi experiences severe air pollution throughout the year due to factors such as vehicle emissions, industrial activities, construction dust, crop residue burning, and weather conditions. This project aims to analyze AQI data to understand pollution patterns and identify the major factors affecting air quality.

3. OBJECTIVES

- Analyze Delhi AQI data.
 - Identify the major pollutants affecting air quality.
 - Study monthly and seasonal AQI variations.
 - Perform statistical analysis of pollutants.
 - Visualize AQI trends using graphs.
 - Provide recommendations to improve air quality.
-

4. RESEARCH QUESTIONS

- How does AQI change over time?
 - Which pollutants contribute the most to AQI?
 - Which months have the highest pollution levels?
 - Is there any relationship between AQI and PM_{2.5} or PM₁₀?
 - What recommendations can reduce air pollution in Delhi?
-

5. DATASET DESCRIPTION

The dataset contains daily air quality information for Delhi.

Common features include:

- Date
- AQI
- PM_{2.5}
- PM₁₀
- NO₂

- SO₂
 - CO
 - O₃
 - City/Location
-

6. TOOLS AND TECHNOLOGIES

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Jupyter Notebook / Google Colab

7. METHODOLOGY

➤ Install required Python libraries.

```
|pip install pandas
|pip install matplotlib
|pip install seaborn

Requirement already satisfied: pandas in c:\users\hp\jupyter\lib\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\hp\jupyter\lib\site-packages (from pandas) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\jupyter\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\jupyter\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\jupyter\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\hp\jupyter\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: matplotlib in c:\users\hp\jupyter\lib\site-packages (3.10.6)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (2.3.5)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\jupyter\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\jupyter\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: seaborn in c:\users\hp\jupyter\lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\hp\jupyter\lib\site-packages (from seaborn) (2.3.5)
Requirement already satisfied: pandas>=1.2 in c:\users\hp\jupyter\lib\site-packages (from seaborn) (2.3.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\hp\jupyter\lib\site-packages (from seaborn) (3.10.6)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\jupyter\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
```

➤ Load the CSV dataset.

```
df = pd.read_csv(r"delhiaqi.csv")
```

➤ Understand dataset structure.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
print(df.head())
```

```
      date      co      no      no2      o3      so2      pm2_5      pm10 \
0  2023-01-01 00:00:00 1655.58   1.66  39.41   5.90  17.88  169.29  194.64
1  2023-01-01 01:00:00 1869.20   6.82  42.16   1.99  22.17  182.84  211.08
2  2023-01-01 02:00:00 2510.07  27.72  43.87   0.02  30.04  220.25  260.68
3  2023-01-01 03:00:00 3150.94  55.43  44.55   0.85  35.76  252.90  304.12
4  2023-01-01 04:00:00 3471.37  68.84  45.24   5.45  39.10  266.36  322.80

      nh3
0    5.83
1    7.66
2   11.40
3   13.55
4   14.19
```

```
print(df.describe())
```

```
      count      co      no      no2      o3      so2 \
count    561.000000  561.000000  561.000000  561.000000  561.000000
mean    3814.942210   51.181979   75.292496   30.141943   64.655936
std     3227.744681   83.904476   42.473791   39.979405   61.073080
min       654.220000    0.000000   13.370000    0.000000    5.250000
25%     1708.980000    3.380000   44.550000    0.070000   28.130000
50%     2590.180000   13.300000   63.750000   11.800000   47.210000
75%     4432.680000   59.010000   97.330000   47.210000   77.250000
max     16876.220000  425.580000  263.210000  164.510000  511.170000

      count      pm2_5      pm10      nh3
count    561.000000  561.000000  561.000000
mean     358.256364  420.988414   26.425062
std      227.359117  271.287026   36.563094
min        60.100000   69.080000    0.630000
25%       204.450000  240.900000    8.230000
50%       301.170000  340.900000   14.820000
75%       416.650000  482.570000   26.350000
max       1310.200000 1499.270000  267.510000
```

➤ Check missing values.

```
print(df.isnull().sum())
```

```
date      0
co        0
no        0
no2       0
o3        0
so2       0
pm2_5     0
pm10      0
nh3       0
dtype: int64
```

- Convert date column into datetime format.

```
print(df.columns)
```

```
Index(['date', 'co', 'no', 'no2', 'o3', 'so2', 'pm2_5', 'pm10', 'nh3'], dtype='object')
```

- Basic Statistics

```
print(df.dtypes)
```

```
date      object
co        float64
no        float64
no2       float64
o3        float64
so2       float64
pm2_5     float64
pm10      float64
nh3       float64
dtype: object
```

```
print(df.duplicated().sum())
```

```
0
```

```
df = df.drop_duplicates()
```

- Create visualizations using Matplotlib and Seaborn

1- HEATMAP:

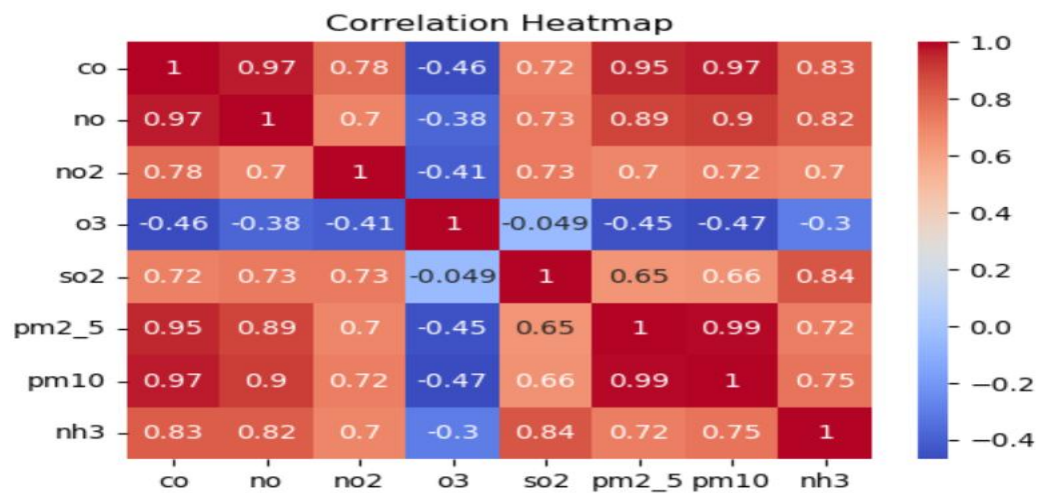
```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

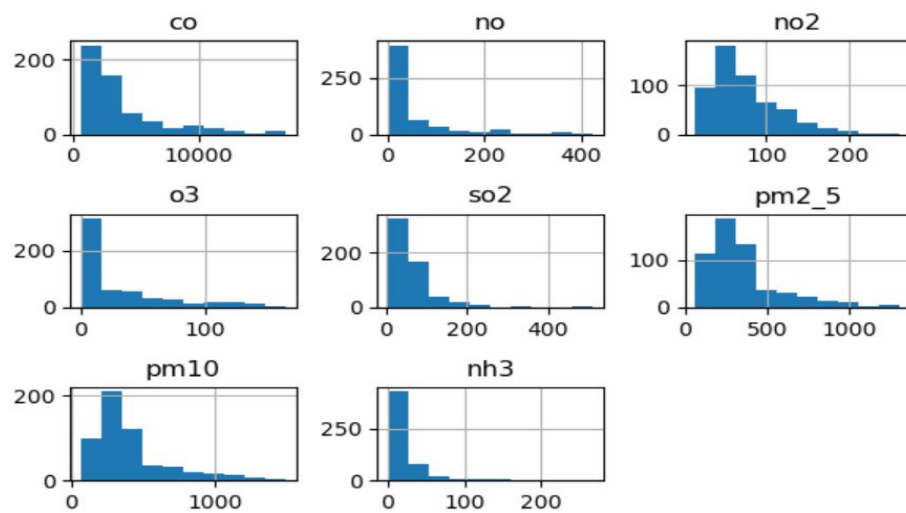
OUTPUT -



2- HISTOGRAM :

```
df.hist(figsize=(6,4))  
plt.tight_layout()  
  
plt.show()
```

OUTPUT -

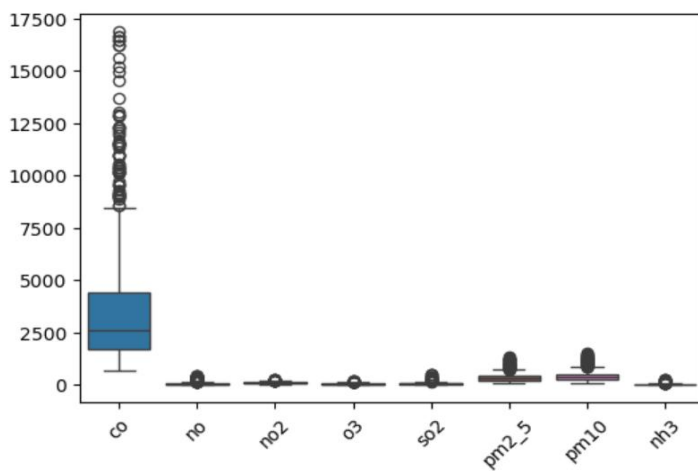


3- BOXPLOT :

```
plt.figure(figsize=(6,4))
sns.boxplot(data=numeric_df)
plt.xticks(rotation=45)

plt.show()
```

OUTPUT -

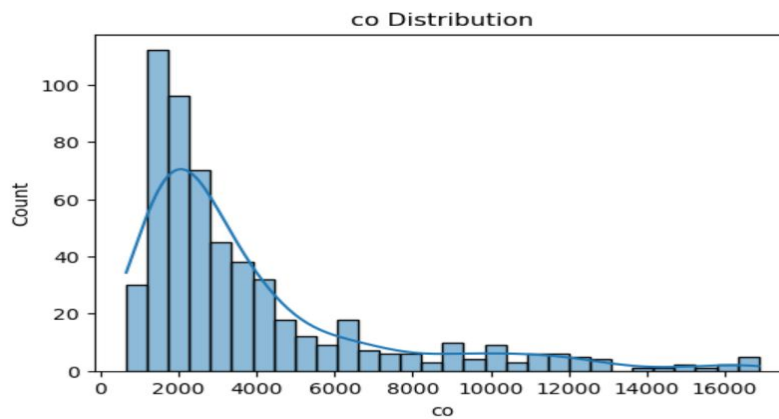


4- HISTPLOT :

```
plt.figure(figsize=(6,4))
sns.histplot(df["co"], bins=30, kde=True)
plt.title("co Distribution")

plt.show()
```

OUTPUT -

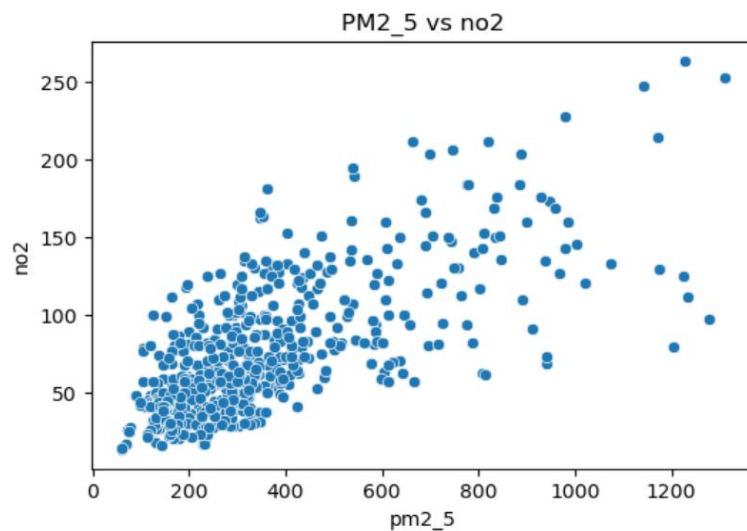


5- SCATTER PLOT :

```
plt.figure(figsize=(6,4))
sns.scatterplot(x="pm2_5", y="no2", data=df)
plt.title("PM2_5 vs no2")

plt.show()
```

OUTPUT -

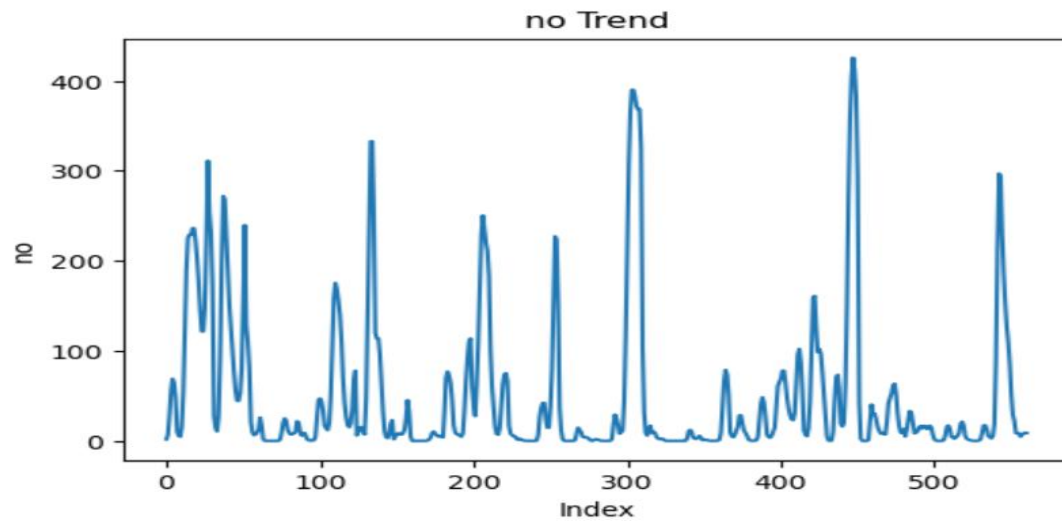


6- PLOT :

```
plt.figure(figsize=(6,4))
plt.plot(df["no"])
plt.title("no Trend")
plt.xlabel("Index")
plt.ylabel("no")

plt.show()
```

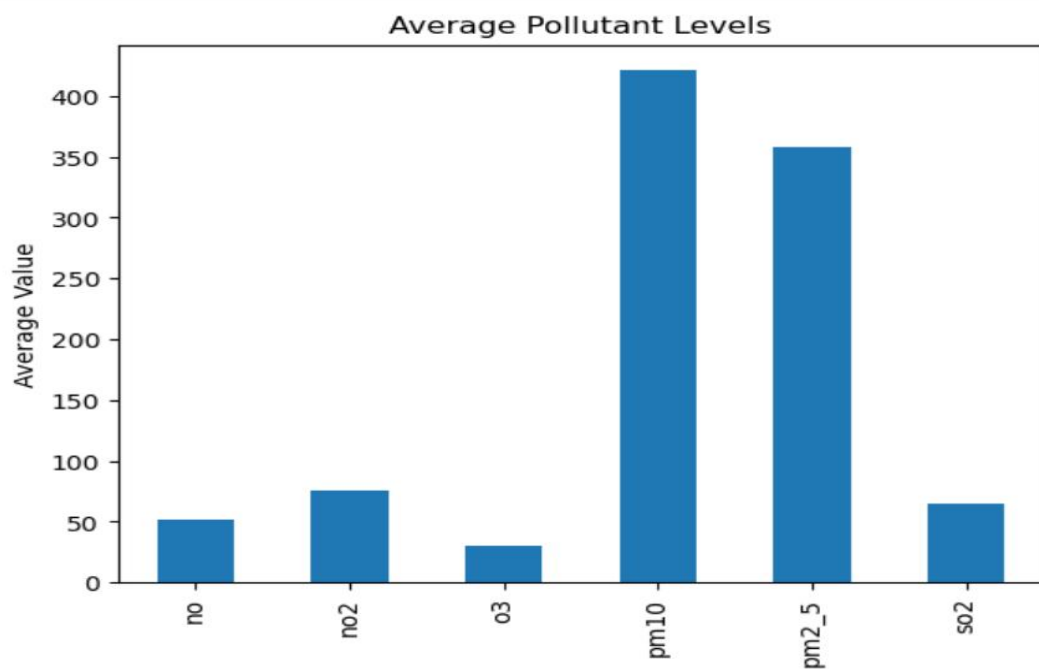
OUTPUT -



7- BAR GRAPH :

```
pollutants = ["no", "no2", "o3", "pm10", "pm2_5", "so2"]  
df[pollutants].mean().plot(kind="bar")  
plt.title("Average Pollutant Levels")  
plt.ylabel("Average Value")  
  
plt.show()
```

OUTPUT –



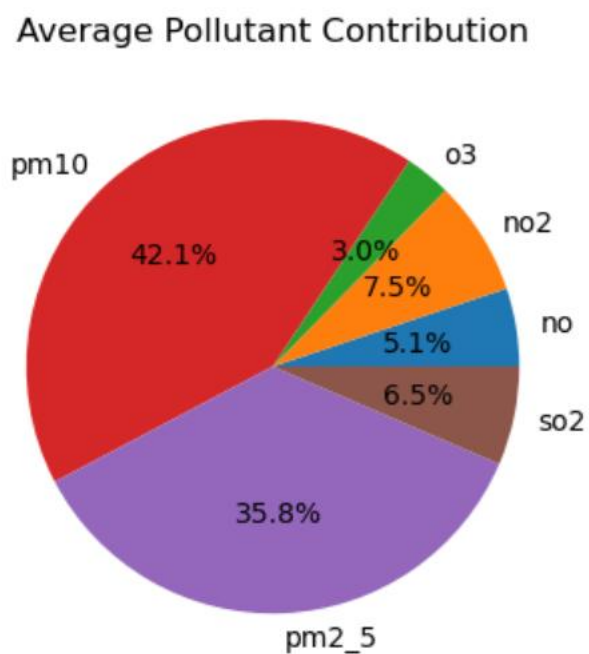
8- PIE CHART :

```
pollutants = ["no", "no2", "o3", "pm10", "pm2_5", "so2"]

df[pollutants].mean().plot(
    kind="pie",
    autopct="%1.1f%%",
    figsize=(4,4)
)
plt.title("Average Pollutant Contribution")
plt.ylabel("")

plt.show()
```

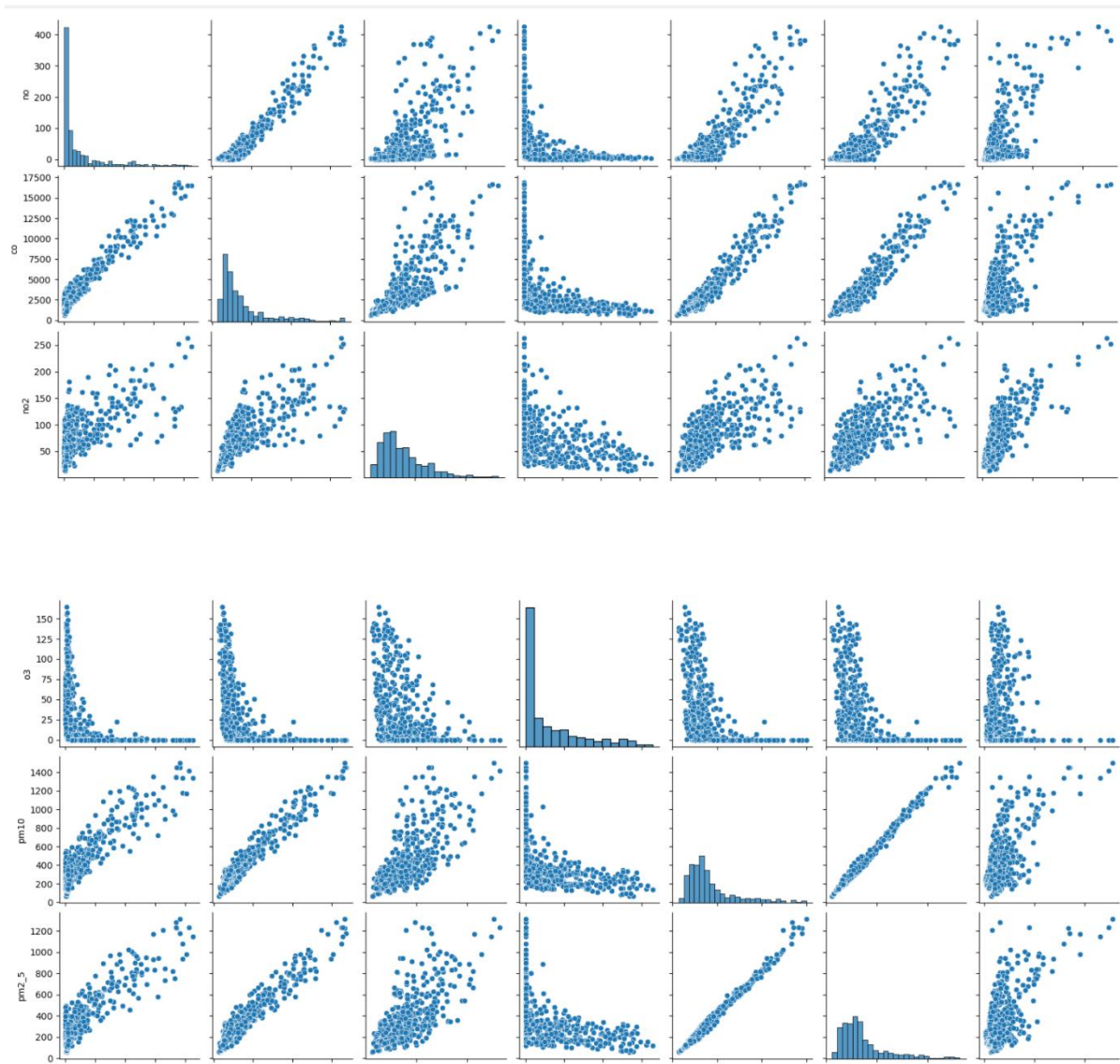
OUTPUT –



9- PAIRPLOT :

```
sns.pairplot(df[["no", "co", "no2", "o3", "pm10", "pm2_5", "so2"]])
plt.figure(figsize=(2,2))
plt.show()
```

OUTPUT –



Conclusion :

- PM_{2.5} and PM₁₀ are the major contributors to poor air quality.
- AQI is highest during winter due to low wind speed and stubble burning.
- Traffic and industrial emissions significantly affect AQI.
- Strong positive correlation exists between AQI and particulate matter.
- Regular monitoring and pollution-control measures can help improve Delhi's air quality.

Future Scope :

The project can be further improved by:

- Building Machine Learning models to predict AQI.
- Developing a real-time AQI monitoring dashboard.
- Integrating live pollution data through APIs.
- Performing location-wise forecasting.
- Creating interactive dashboards using Power BI or Tableau.
- Deploying the model as a web application using Streamlit.

Limitations :

- The analysis is based only on the available dataset.
- Missing values may affect accuracy.
- External factors such as weather conditions and traffic were not fully considered.
- Real-time AQI prediction was not included.

References :

1. Central Pollution Control Board (CPCB)
2. Delhi Pollution Control Committee (DPCC)
3. Kaggle AQI Datasets
4. Python Documentation
5. Pandas Documentation
6. NumPy Documentation
7. Matplotlib Documentation
8. Seaborn Documentation
9. Scikit-learn Documentation